

claude-windows / 6e9d0958-a2ec-4a8e-bb2d-6cce9833c527

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\subagents\agent-aba1b1bb1014c1eef.jsonl`
- SHA-256: `215203962a6c7df25c12f508b2bceab65195a3c54269e086c34913897eae41cc`
- Source modified: `2026-07-04T00:45:42+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `subagents`
- session_id: `6e9d0958-a2ec-4a8e-bb2d-6cce9833c527`

Transcript

1. user (2026-07-04T00:44:52.049Z)

Code-review finder. Target: PR #458 of badminton-highlight-indexer. Diff: `cd C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer && gh pr diff 458`. PR branch = `origin/codex/a7-a9-quality-chain`. Files at scratchpad/pr458_fusion_golden.py, pr458_fusion_audio.py.

ANGLE ? efficiency. Flag wasted work; name the cheaper form:

1. `is\_full\_fusion\_feature\_file(out)` opens+json.loads the ENTIRE feature JSON for every video in run()'s loop, just to check schema ? these JSONs can be large (per-window shuttle/person/audio features for a full match). Called once per spec. Is it reading a big file only to inspect the first candidate's keys? Could it check cheaply (e.g. the file is only read to classify; the extract then re-reads nothing ? but for a full-fusion file it's read once to skip). Acceptable, or is it re-read elsewhere (double read)?
2. `attach\_source\_video\_paths` ? `load\_source\_video\_paths` re-reads + re-parses the source manifest each call; run() and augment() each call it once (fine). But is it called per-spec anywhere (inside a loop)?
3. `\_filter\_specs` builds `wanted`, `wanted\_set`, `filtered`, `found`, `missing` ? any $O(n^2)$ over specs?
4. Any redundant os.path.isfile / file stat repeated per spec that could be hoisted? Only flag measurable waste (big-file reads in loops count; nanosecond stuff doesn't). Return up to 6 candidates as JSON [{file, line, summary, failure_scenario}]. [] if none. Final message = only the JSON array.

2. assistant (2026-07-04T00:44:54.024Z)

I'll analyze the PR for efficiency issues in the specified files. Let me start by examining the diff and then reviewing the code.

3. user (2026-07-04T00:44:55.772Z)

SESSION_HANDOFF.md
backend/eval/fusion_audio.py
backend/eval/fusion_golden.py
docs/ALPHA_LAUNCH_READINESS.md
tests/test_ablation.py
tests/test_audio_features.py
tests/test_fusion_compare.py

4. assistant (2026-07-04T00:44:57.571Z)

Now let me read the two files mentioned:

5. user (2026-07-04T00:44:58.429Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\khelsutra-

guru\badminton-highlight-indexer.

6. user (2026-07-04T00:44:58.491Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\kheIsutra-guru\badminton-highlight-indexer.

7. assistant (2026-07-04T00:45:00.036Z)

Let me check the actual files from the PR diff:

8. user (2026-07-04T00:45:00.474Z)

```
1      """Offline golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.
2
3      Mirrors `distill_local`'s CSV-driven flow but ADDS the ~5 person-cue features per
shuttle
4      candidate window, by running `PersonDetector` over the ORIGINAL video. It reuses the
5      EXISTING golden trajectory CSVs (so **no WASB re-run**), needs **no Gemini and no DB**,
and
6      writes one small JSON per video. The LOVO comparison (`fusion_compare.py`) then runs
100% on
7      CPU offline. So the GPU touches exactly one thing ? person detection ? and the lift is
8      measured by the shipped experiment harness; person features enter only as feature
columns the
9      learned scorer weights (no special-casing, default-OFF until the golden ?F1 justifies
them).
10
11      Run (on the GPU box):
12      python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json
--out-dir output
13
14      Each output `output/<name>_golden_features.json` is the replayable feature substrate ?
re-run
15      `fusion_compare` with different variants/thresholds forever without re-detecting on the
GPU.
16      """
17
18      from __future__ import annotations
19
20      import argparse
21      import json
22      import os
23      import time
24      from typing import Any, Dict, List, Optional, Sequence
25
26      from backend.eval import distill_local
27      from backend.eval.calibrate_wasb import load_golden_gts
28      from backend.eval.golden_manifest import load_resolved_manifest
29      from backend.eval.training import label_candidates
30      from backend.pipeline.detectors import fusion_features as ff
31      from backend.pipeline.detectors import rally_gate
32      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
33      from backend.utils.run_telemetry import RunTelemetry
34
35      # The 5 fps/resolution-invariant shuttle features distill_local already computes.
36      SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
37      # The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
38      PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
39      SOURCE_MANIFEST_ENV = "RALLY_GOLDEN_SOURCE_MANIFEST"
40
41
42      def is_full_fusion_feature_file(path: str) -> bool:
43          """True when an existing golden feature JSON already has shuttle+person cue dicts.
```

```

44
45     Corpus-growth runs can leave leaner windowing-only JSONs with the same filename
pattern.
46     Those are useful for gts/windowing analysis, but they are not the replayable fusion
47     substrate that ``fusion_compare`` and ``ablation`` need.
48     """
49
50     try:
51         with open(path, encoding="utf-8") as f:
52             data = json.load(f)
53     except (OSError, json.JSONDecodeError, TypeError):
54         return False
55     candidates = data.get("candidates")
56     if not isinstance(candidates, list):
57         return False
58     if not candidates:
59         return True
60     return all(
61         isinstance(candidate, dict)
62         and "shuttle" in candidate
63         and "person" in candidate
64         for candidate in candidates
65     )
66
67
68 def load_source_video_paths(source_manifest_path: Optional[str]) -> Dict[str, str]:
69     """Load ``video_id -> local_path`` from the collector golden-source manifest.
70
71     The collector records source/proxy ownership and full-file MD5s. Only entries marked
72     ``present_local`` with a concrete ``local_path`` can drive local GPU extraction.
73     """
74
75     if not source_manifest_path:
76         return {}
77     with open(source_manifest_path, encoding="utf-8") as f:
78         data = json.load(f)
79     entries = data.get("videos", data) if isinstance(data, dict) else data
80     if not isinstance(entries, list):
81         raise ValueError(f"source manifest must contain a videos list: {source_manifest_path}")
82
83     paths: Dict[str, str] = {}
84     for entry in entries:
85         if not isinstance(entry, dict):
86             continue
87         video_id = entry.get("video_id") or entry.get("name")
88         local_path = entry.get("local_path")
89         if not video_id or not local_path:
90             continue
91         if entry.get("source_status", "present_local") != "present_local":
92             continue
93         paths[str(video_id)] = str(local_path)
94     return paths
95
96
97 def attach_source_video_paths(
98     specs: Sequence[Dict[str, Any]], source_manifest_path: Optional[str]
99 ) -> int:
100     """Attach explicit ``video_path`` fields to manifest specs from the source manifest.
101
102     Returns the number of specs updated. The collector source manifest is authoritative
103     for local source/proxy video location and overrides derived/stale manifest paths.
104     """
105
106     source_paths = load_source_video_paths(source_manifest_path)

```

```

107         updated = 0
108     for spec in specs:
109         name = str(spec.get("name", ""))
110         source_path = source_paths.get(name)
111         if not source_path:
112             continue
113         if spec.get("video_path") == source_path:
114             continue
115         spec["video_path"] = source_path
116         updated += 1
117     return updated
118
119
120     def _filter_specs(
121         specs: Sequence[Dict[str, Any]], only: Sequence[str]
122     ) -> List[Dict[str, Any]]:
123         if not only:
124             return list(specs)
125         wanted = list(dict.fromkeys(only))
126         wanted_set = set(wanted)
127         filtered = [spec for spec in specs if str(spec.get("name")) in wanted_set]
128         found = {str(spec.get("name")) for spec in filtered}
129         missing = [name for name in wanted if name not in found]
130         if missing:
131             raise ValueError(f"--only requested unknown video(s): {' '.join(missing)}")
132         return filtered
133
134
135     def _derive_video_path(spec: Dict[str, Any]) -> str:
136         """Original MP4 for a manifest entry ? explicit `video_path`, else derived from the
137         golden labels path (`X.rallies.csv` -> `X.MP4`, the golden-set naming
138         convention)."""
139         if spec.get("video_path"):
140             return str(spec["video_path"])
141         labels = str(spec["labels"])
142         if labels.endswith(".rallies.csv"):
143             return labels[: -len(".rallies.csv")] + ".MP4"
144         raise ValueError(
145             f"{spec.get('name')}: no video_path and labels {labels!r} isn't *.rallies.csv"
146         )
147
148     def extract_video(
149         spec: Dict[str, Any],
150         detector: Optional[Any] = None,
151         frame_skip: int = 3,
152         iou: float = 0.5,
153         log_every_sec: float = 15.0,
154         telemetry: Optional[Any] = None,
155         use_gpu: bool = True,
156         detector_model: Optional[str] = None,
157         max_gpu_temp_c: Optional[float] = None,
158     ) -> Dict[str, Any]:
159         """One golden video -> a feature record (shuttle + person features per candidate
160         window).
161
162         Returns ``{name, fps, frame_width,
163         candidates:[{start,end,shuttle{ },person{ },label}], gts}``.
164         `detector` (anything with `detections_over_windows`) is injectable for GPU-free
165         tests;
166         when None a real `PersonDetector` is built (loads torchvision on first use).
167         `telemetry` (a `RunTelemetry` or None) records per-frame velocity + machine-health
168         snapshots
169         from the detection progress callback so a hard kill leaves a forensic runlog.
170         """

```

```

167     fps, fw = float(spec["fps"]), float(spec["frame_width"])
168     traj = str(spec["trajectory"])
169     points = TrackNetRunner.parse_trajectory_csv(traj)
170     intervals, x_shuttle = distill_local.build_candidates(traj, fps, fw)
171     gts = load_golden_gts(str(spec["labels"]))
172     labels = label_candidates(intervals, gts, iou)
173     # Net axis for the person two-sided split ? reuse rally_gate's camera-agnostic
estimate
174     # (parity with the shuttle net-crossing gate) rather than hard-coding an
orientation.
175     axis, net_px, _span = rally_gate.estimate_play_axis(points)
176     video_path = _derive_video_path(spec)
177     if not os.path.isfile(video_path):
178         raise FileNotFoundError(
179             f"{spec.get('name')}: source video not found for fusion extraction:
{video_path}"
180         )
181
182     if detector is None:
183         from backend.pipeline.detectors.person_detector import PersonDetector
184
185         indexing_cfg: Dict[str, Any] = {"use_gpu": use_gpu}
186         if detector_model:
187             indexing_cfg["detector_model"] = detector_model
188         detector = PersonDetector({"indexing": indexing_cfg})
189
190     # Single-pass person detection over ALL candidate windows (open the video ONCE, read
191     # forward ? no per-window seek). Then compute features per window (pure, no GPU).
192     n = len(intervals)
193     win_frames = [(round(s * fps), round(e * fps)) for (s, e) in intervals]
194     print(
195         f"[fusion_golden]   {spec['name']}: {n} windows ? person-detecting in one
pass...",
196         flush=True,
197     )
198     t0 = time.monotonic()
199     state = {"last": t0}
200
201     def _progress(done: int, total: int) -> None:
202         rec = None
203         if telemetry is not None:
204             rec = telemetry.snapshot(done, total, phase=spec["name"])
205         if max_gpu_temp_c is not None and rec is not None:
206             gpu = rec.get("gpu") or {}
207             temp = gpu.get("temp_c")
208             if temp is not None and float(temp) >= max_gpu_temp_c:
209                 raise RuntimeError(
210                     f"{spec['name']}: GPU temperature {float(temp):.1f}C reached "
211                     f"--max-gpu-temp-c {max_gpu_temp_c:.1f}; aborting before thermal
shutdown"
212                 )
213         now = time.monotonic()
214         if now - state["last"] >= log_every_sec or done >= total:
215             el = now - t0
216             rate = done / el if el > 0 else 0.0
217             eta = (total - done) / rate if rate > 0 else 0.0
218             pct = 100 * done // total if total else 0
219             print(
220                 f"[fusion_golden]   {spec['name']}: detect frame {done}/{total} ({pct}%)
"
221                 f"| {el:.0f}s elapsed | ETA {eta:.0f}s",
222                 flush=True,
223             )
224             state["last"] = now
225

```

```

226 per_window = detector.detections_over_windows(
227     video_path, win_frames, frame_skip, progress_cb=progress
228 )
229
230 candidates: List[Dict[str, Any]] = []
231 for i, ((s, e), xs) in enumerate(zip(intervals, x_shuttle)):
232     det = per_window[i]
233     det_fw = det.get("frame_width") or fw
234     det_fh = det.get("frame_height") or 0.0
235     # Normalise the (pixel) net coord to 0-1 on the play axis, matching the foot-
point
236     # normalisation fusion_features uses (x/width, y/height).
237     if axis == "x":
238         net_norm = net_px / det_fw if det_fw > 0 else 0.5
239     else:
240         net_norm = net_px / det_fh if det_fh > 0 else 0.5
241     sf, ef = win_frames[i]
242     shuttle_in = [p for p in points if sf <= p.frame <= ef]
243     person = ff.compute_person_features(
244         det["frames"],
245         shuttle_in,
246         det_fw,
247         det_fh,
248         court_polygon=None,
249         net=(axis, net_norm),
250     )
251     candidates.append(
252         {
253             "start": float(s),
254             "end": float(e),
255             "shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
256             "person": {k: float(v) for k, v in person.items()},
257             "label": int(labels[i]),
258         }
259     )
260     return {
261         "name": spec["name"],
262         "fps": fps,
263         "frame_width": fw,
264         "candidates": candidates,
265         "gts": [[float(a), float(b)] for a, b in gts],
266     }
267
268
269 def run(
270     manifest_path: str,
271     out_dir: str,
272     frame_skip: int = 3,
273     iou: float = 0.5,
274     fresh: bool = False,
275     smallest_first: bool = False,
276     source_manifest: Optional[str] = None,
277     only: Sequence[str] = (),
278     missing_fusion_only: bool = False,
279     use_gpu: bool = True,
280     detector_model: Optional[str] = None,
281     max_gpu_temp_c: Optional[float] = None,
282 ) -> List[str]:
283     specs = load_resolved_manifest(manifest_path)
284     source_manifest = source_manifest or os.environ.get(SOURCE_MANIFEST_ENV)
285     if source_manifest:
286         updated = attach_source_video_paths(specs, source_manifest)
287         print(
288             f"[fusion_golden] attached {updated}/{len(specs)} source video path(s) "
289             f"from {source_manifest}",

```

```

290         flush=True,
291     )
292     specs = _filter_specs(specs, only)
293     if smallest_first:
294         # Process small videos first for quick feedback (the slow full match lands
last);
295         # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
row/frame).
296         specs = sorted(
297             specs,
298             key=lambda s: (
299                 os.path.getsize(s["trajectory"])
300                 if os.path.isfile(s.get("trajectory", ""))
301                 else 0
302             ),
303         )
304     os.makedirs(out_dir, exist_ok=True)
305     # Run-telemetry black box: a long GPU run can be OS-killed with no traceback, so
keep a tiny
306     # rotating runlog of velocity + machine health for the post-mortem. Constructing it
must never
307     # break extraction, so guard the build and fall back to no telemetry on any error.
308     telem: Optional[RunTelemetry]
309     try:
310         telem = RunTelemetry(
311             os.path.join(out_dir, "run_telemetry.jsonl"), label="fusion_golden"
312         )
313     except Exception: # noqa: BLE001 - telemetry wiring must never break the run it
observes
314         telem = None
315     written: List[str] = []
316     for vi, spec in enumerate(specs, 1):
317         out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
318         existing = os.path.isfile(out)
319         existing_full = existing and is_full_fusion_feature_file(out)
320         # Resumable: skip a video whose full-fusion JSON already exists. A leaner
321         # windowing-only JSON is stale for A7, so re-extract it even without --fresh.
322         if missing_fusion_only and not fresh and existing_full:
323             print(
324                 f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: full-fusion "
325                 f"features already present -> {out}",
326                 flush=True,
327             )
328             written.append(out)
329             continue
330         if not fresh and existing and existing_full:
331             print(
332                 f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already extracted
"
333                 f"-> {out} (use --fresh to redo)",
334                 flush=True,
335             )
336             written.append(out)
337             continue
338         if not fresh and existing and not existing_full:
339             print(
340                 f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: existing JSON "
341                 f"is not full-fusion schema; re-extracting...",
342                 flush=True,
343             )
344             print(
345                 f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
346                 flush=True,
347             )
348         t0 = time.monotonic()

```

```

349         if telem is not None:
350             telem.event("video_start", spec["name"])
351         try:
352             data = extract_video(
353                 spec,
354                 frame_skip=frame_skip,
355                 iou=iou,
356                 telemetry=telem,
357                 use_gpu=use_gpu,
358                 detector_model=detector_model,
359                 max_gpu_temp_c=max_gpu_temp_c,
360             )
361         except Exception as e: # noqa: BLE001 - record machine state at death, then re-
raise
362             if telem is not None:
363                 telem.event("error", spec["name"], err=str(e))
364             raise
365         n_pos = sum(c["label"] for c in data["candidates"])
366         with open(out, "w", encoding="utf-8") as f:
367             json.dump(data, f, indent=2)
368         if telem is not None:
369             telem.event(
370                 "video_done",
371                 spec["name"],
372                 windows=len(data["candidates"]),
373                 positives=n_pos,
374             )
375         print(
376             f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: DONE in
{time.monotonic() - t0:.0f}s "
377             f"| {len(data['candidates'])} windows ({n_pos} positive) |
{len(data['gts'])} golden rallies "
378             f"-> {out}",
379             flush=True,
380         )
381         written.append(out)
382     return written
383
384
385 def main() -> None:
386     ap = argparse.ArgumentParser(
387         description="Extract golden-set shuttle+person features (fusion P3 GPU step)."
388     )
389     ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
390     ap.add_argument("--out-dir", default="output")
391     ap.add_argument("--frame-skip", type=int, default=3)
392     ap.add_argument("--iou", type=float, default=0.5)
393     ap.add_argument(
394         "--fresh", action="store_true", help="re-extract even if a JSON already exists"
395     )
396     ap.add_argument(
397         "--smallest-first",
398         action="store_true",
399         help="process smaller videos first for quick feedback (LOVO is order-
independent)",
400     )
401     ap.add_argument(
402         "--source-manifest",
403         default=None,
404         help=(
405             "collector golden_source_videos.json; overrides video paths for source/proxy
"
406             f"videos (or set {SOURCE_MANIFEST_ENV})"
407         ),
408     )

```



```

409     ap.add_argument(
410         "--only",
411         action="append",
412         default=[],
413         metavar="VIDEO_ID",
414         help="process one manifest video by name; repeatable",
415     )
416     ap.add_argument(
417         "--missing-fusion-only",
418         action="store_true",
419         help="skip videos whose existing JSON already has full shuttle+person fusion
schema",
420     )
421     ap.add_argument(
422         "--cpu",
423         action="store_true",
424         help="run torchvision person detection on CPU instead of CUDA",
425     )
426     ap.add_argument(
427         "--detector-model",
428         default=None,
429         choices=[
430             "fasterrcnn_resnet50_fpn",
431             "fasterrcnn_mobilenet_v3_large_fpn",
432             "retinanet_resnet50_fpn",
433         ],
434         help="torchvision detector model; default keeps the configured Faster R-CNN
ResNet",
435     )
436     ap.add_argument(
437         "--max-gpu-temp-c",
438         type=float,
439         default=None,
440         help="abort extraction if telemetry sees GPU temperature at or above this
value",
441     )
442     args = ap.parse_args()
443     run(
444         args.manifest,
445         args.out_dir,
446         args.frame_skip,
447         args.iou,
448         fresh=args.fresh,
449         smallest_first=args.smallest_first,
450         source_manifest=args.source_manifest,
451         only=args.only,
452         missing_fusion_only=args.missing_fusion_only,
453         use_gpu=not args.cpu,
454         detector_model=args.detector_model,
455         max_gpu_temp_c=args.max_gpu_temp_c,
456     )
457
458
459     if __name__ == "__main__":
460         main()
461

```

9. user (2026-07-04T00:45:00.586Z)

```

1     """Fusion P4 ? measure the INCREMENTAL audio-cue lift on top of the person cue (CPU-
only).
2
3     Reuses the person-feature JSONs from `fusion_golden.py` (no GPU re-detect): `augment`
runs
4     `detect_hits_in_video` once per video (ffmpeg + numpy, CPU) and adds the 3 audio

```

```

features per
5      window; `compare_increment` then runs the SHIPPED harness between a shuttle+person
variant
6      and a shuttle+person+audio variant ? so audio is measured as a marginal contribution ON
TOP
7      of person, not against raw shuttle. Audio enters only as columns the learned scorer
weights
8      (no special-casing, default-OFF); the golden-set ?F1 decides whether it earns its keep.
9
10     Run (no GPU):
11         python -m backend.eval.fusion_audio --manifest output/human_lovo_manifest.json
--features-dir output --augment --compare
12     """
13
14     from __future__ import annotations
15
16     import argparse
17     import json
18     import logging
19     import os
20     from typing import Any, Dict, List, Optional, Sequence
21
22     from backend.eval import experiment
23     from backend.eval.fusion_compare import delta_stats
24     from backend.eval.fusion_golden import (
25         PERSON_FEATURE_NAMES,
26         SHUTTLE_FEATURE_NAMES,
27         SOURCE_MANIFEST_ENV,
28         _derive_video_path,
29         _filter_specs,
30         attach_source_video_paths,
31     )
32     from backend.eval.golden_manifest import load_resolved_manifest
33     from backend.pipeline.audio.hit_detector import detect_hits_in_video
34     from backend.pipeline.detectors import audio_features as af
35
36     logger = logging.getLogger(__name__)
37
38
39     def augment(
40         manifest_path: str,
41         features_dir: str,
42         k: float = 2.5,
43         source_manifest: Optional[str] = None,
44         only: Sequence[str] = (),
45     ) -> List[str]:
46         """Add the 3 audio features to each existing ``<name>_golden_features.json``
window."""
47         specs = load_resolved_manifest(manifest_path)
48         source_manifest = source_manifest or os.environ.get(SOURCE_MANIFEST_ENV)
49         if source_manifest:
50             updated = attach_source_video_paths(specs, source_manifest)
51             print(
52                 f"[fusion_audio] attached {updated}/{len(specs)} source video path(s) "
53                 f"from {source_manifest}",
54                 flush=True,
55             )
56         specs = _filter_specs(specs, only)
57         written: List[str] = []
58         for spec in specs:
59             jp = os.path.join(features_dir, f"{spec['name']}_golden_features.json")
60             if not os.path.isfile(jp):
61                 print(
62                     f"[fusion_audio] {spec['name']}: no feature JSON ? run fusion_golden
first; skipping"

```

```

63         )
64         continue
65         video = _derive_video_path(spec)
66         if not os.path.isfile(video):
67             raise FileNotFoundError(
68                 f"{spec.get('name')}: source video not found for audio augmentation:
{video}"
69             )
70         print(
71             f"[fusion_audio] {spec['name']}: detecting audio hits (k={k})...",
72             flush=True,
73         )
74         hits = detect_hits_in_video(video, k=k)
75         with open(jp, encoding="utf-8") as f:
76             data = json.load(f)
77             for c in data["candidates"]:
78                 c["audio"] = af.compute_audio_features(hits, c["start"], c["end"])
79                 with open(jp, "w", encoding="utf-8") as f:
80                     json.dump(data, f, indent=2)
81             print(
82                 f"[fusion_audio] {spec['name']}: {len(hits)} hits over
{len(data['candidates'])} windows",
83                 flush=True,
84             )
85             written.append(jp)
86         return written
87
88
89     def load_videos_with_audio(features_dir: str) -> List[experiment.VideoCandidates]:
90         """Load the FULL-FUSION-SCHEMA feature JSONs into VideoCandidates including the
audio
91         columns (requires `augment` to have run ? raises if a fusion JSON lacks audio).
92
93         Leaner velocity-windowing-only files (no ``shuttle``/``person`` cue dicts) are
SKIPPED, not
94         crashed on, so a mixed corpus loads the fusion subset cleanly (see
``_is_fusion_schema``).
95         The skip is checked BEFORE the audio-presence guard, so a genuine fusion file that
simply
96         hasn't been ``--augment``-ed still raises rather than being silently dropped."""
97         return experiment.load_golden_videos(
98             features_dir,
99             shuttle_columns=SHUTTLE_FEATURE_NAMES,
100             person_columns=PERSON_FEATURE_NAMES,
101             audio_columns=af.AUDIO_FEATURE_NAMES,
102             audio_policy="require",
103             skip_logger=logger,
104             skip_message="fusion_audio.load_videos_with_audio: skipping %s ? windowing-only
"
105             "schema (no shuttle/person cue dicts), not a fusion-feature file",
106             no_audio_error="{path} has no audio features ? run ``--augment`` first",
107         )
108
109
110     def compare_incremental(
111         videos: List[experiment.VideoCandidates], iou: float = 0.5, threshold: float = 0.5
112     ) -> Dict[str, Any]:
113         """Incremental audio lift: shuttle+person vs shuttle+person+audio (honest
LOVO)."""
114         person = experiment.Variant(
115             name="shuttle_person",
116             feature_names=list(SHUTTLE_FEATURE_NAMES) + list(PERSON_FEATURE_NAMES),
117             threshold=threshold,
118         )
119         person_audio = experiment.Variant(

```

```

120         name="shuttle_person_audio",
121         feature_names=list(SHUTTLE_FEATURE_NAMES)
122         + list(PERSON_FEATURE_NAMES)
123         + list(af.AUDIO_FEATURE_NAMES),
124         threshold=threshold,
125     )
126     return experiment.compare(videos, person, person_audio, iou=iou, honest=True)
127
128
129     def format_result(res: Dict[str, Any], iou: float) -> str:
130         a, b, d = res["variant_a"]["aggregate"], res["variant_b"]["aggregate"], res["delta"]
131         ds = delta_stats(res)
132         st = ds["sign_test"]
133         lines = [
134             f"=== Fusion P4 incremental AUDIO LOVO (n={res['num_videos']}, IoU>={iou},
135             honest) ===",
136             f"    shuttle+person          : F1={a['f1']:.3f}  P={a['precision']:.3f}
137             R={a['recall']:.3f}",
138             f"    shuttle+person+audio : F1={b['f1']:.3f}  P={b['precision']:.3f}
139             R={b['recall']:.3f}",
140             f"    incremental ?F1 = {d['f1']:+.3f}  ({res['delta_pct']['f1']:+.1f}%)",
141             f"    paired ?F1 {ds['mean_delta']:+.3f}  [95% CI {ds['ci_low']:+.3f},
142             {ds['ci_high']:+.3f}]  "
143             f"| sign test {int(st['wins'])}W/{int(st['losses'])}L (p={st['p_value']:.3f})  "
144             f"| CI clears 0: {ds['ci_excludes_zero']}",
145         ]
146         for pv in res["per_video_delta"]:
147             lines.append(f"    {pv['video']:<26s} ?F1={pv['f1']:+.3f}")
148         return "\n".join(lines)
149
150     def main() -> None:
151         ap = argparse.ArgumentParser(
152             description="Fusion P4 ? incremental audio lift on the golden set."
153         )
154         ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
155         ap.add_argument("--features-dir", default="output")
156         ap.add_argument("--k", type=float, default=2.5, help="hit-detector sensitivity")
157         ap.add_argument(
158             "--source-manifest",
159             default=None,
160             help=(
161                 "collector golden_source_videos.json; overrides video paths for source/proxy
162                 "
163                 f"videos (or set {SOURCE_MANIFEST_ENV})"
164             ),
165         )
166         ap.add_argument(
167             "--only",
168             action="append",
169             default=[],
170             metavar="VIDEO_ID",
171             help="augment one manifest video by name; repeatable",
172         )
173         ap.add_argument(
174             "--augment",
175             action="store_true",
176             help="add audio features to the existing JSONs",
177         )
178         ap.add_argument(
179             "--compare", action="store_true", help="report the incremental audio LOVO ?F1"
180         )
181         ap.add_argument("--iou", type=float, default=0.5)
182         ap.add_argument("--record", action="store_true")
183         args = ap.parse_args()

```

```

180
181         if args.augment:
182             augment(
183                 args.manifest,
184                 args.features_dir,
185                 args.k,
186                 source_manifest=args.source_manifest,
187                 only=args.only,
188             )
189         if args.compare:
190             videos = load_videos_with_audio(args.features_dir)
191             if len(videos) < 2:
192                 raise SystemExit(f"need >=2 augmented feature files in {args.features_dir}")
193             res = compare_increment(videos, iou=args.iou)
194             print(format_result(res, args.iou))
195             if args.record:
196                 row = experiment.record_experiment(res)
197                 print(
198                     f"recorded -> eval_baselines/experiment_leaderboard.json
199                     ({row['timestamp']})"
200                 )
201
202     if __name__ == "__main__":
203         main()
204

```

10. assistant (2026-07-04T00:45:02.588Z)

Now let me analyze these files for efficiency issues based on the angle specified. Let me examine each point:

11. user (2026-07-04T00:45:03.970Z)

Found 1 file
backend\eval\fusion_golden.py

12. user (2026-07-04T00:45:04.249Z)

Found 2 files
backend\eval\fusion_golden.py
backend\eval\fusion_audio.py

13. user (2026-07-04T00:45:04.584Z)

Found 2 files
backend\eval\fusion_golden.py
backend\eval\fusion_audio.py

14. assistant (2026-07-04T00:45:06.567Z)

Now let me check the full usage patterns in the codebase:

15. user (2026-07-04T00:45:07.167Z)

```

tests\test_fusion_compare.py:345:     assert fusion_golden.is_full_fusion_feature_file(
backend\eval\fusion_golden.py:42: def is_full_fusion_feature_file(path: str) -> bool:
backend\eval\fusion_golden.py:319:         existing_full = existing and
is_full_fusion_feature_file(out)

```

16. user (2026-07-04T00:45:08.001Z)

```

backend/eval/fusion_golden.py:177:     if not os.path.isfile(video_path):
backend/eval/fusion_golden.py:299:         os.path.getsize(s["trajectory"])

```

```

backend/eval/fusion_golden.py:300:         if os.path.isfile(s.get("trajectory", ""))
backend/eval/fusion_golden.py:318:         existing = os.path.isfile(out)
backend/eval/fusion_audio.py:60:         if not os.path.isfile(jp):
backend/eval/fusion_audio.py:66:         if not os.path.isfile(video):

```

17. assistant (2026-07-04T00:45:09.837Z)

Let me examine line 299-300 more carefully and check for other potential redundant operations:

18. user (2026-07-04T00:45:10.509Z)

```

296-         specs = sorted(
297-             specs,
298-             key=lambda s: (
299-                 os.path.getsize(s["trajectory"])
300-                 if os.path.isfile(s.get("trajectory", ""))
301-                 else 0
302-             ),
303-         )

```

19. assistant (2026-07-04T00:45:12.562Z)

Now let me check if `attach\_source\_video\_paths` and/or `load\_source\_video\_paths` are called multiple times per spec or in loops:

20. user (2026-07-04T00:45:13.108Z)

```

63-         and "person" in candidate
64-         for candidate in candidates
65-     )
66-
67-
68:def load_source_video_paths(source_manifest_path: Optional[str]) -> Dict[str, str]:
69-     """Load ``video_id -> local_path`` from the collector golden-source manifest.
70-
71-     The collector records source/proxy ownership and full-file MD5s. Only entries marked
72-     ``present_local`` with a concrete ``local_path`` can drive local GPU extraction.
73-     """
74-
75-     --
76-
77-     continue
78-     paths[str(video_id)] = str(local_path)
79-
80-     return paths
81-
82-
83-
84-
85-
86-
87-
88-
89-
90-
91-
92-
93-
94-
95-
96-
97:def attach_source_video_paths(
98-    specs: Sequence[Dict[str, Any]], source_manifest_path: Optional[str]
99-) -> int:
100-    """Attach explicit ``video_path`` fields to manifest specs from the source manifest.
101-
102-    Returns the number of specs updated. The collector source manifest is authoritative
103-    for local source/proxy video location and overrides derived/stale manifest paths.
104-    """
105-
106-    source_paths = load_source_video_paths(source_manifest_path)
107-    updated = 0
108-    for spec in specs:
109-        name = str(spec.get("name", ""))
110-        source_path = source_paths.get(name)
111-        if not source_path:
112-
113-            --
114-
115-
116-
117-
118-
119-
120-
121-
122-
123-
124-
125-
126-
127-
128-
129-
130-
131-
132-
133-
134-
135-
136-
137-
138-
139-
140-
141-
142-
143-
144-
145-
146-
147-
148-
149-
150-
151-
152-
153-
154-
155-
156-
157-
158-
159-
160-
161-
162-
163-
164-
165-
166-
167-
168-
169-
170-
171-
172-
173-
174-
175-
176-
177-
178-
179-
180-
181-
182-
183-
184-
185-
186-
187-
188-
189-
190-
191-
192-
193-
194-
195-
196-
197-
198-
199-
200-
201-
202-
203-
204-
205-
206-
207-
208-
209-
210-
211-
212-
213-
214-
215-
216-
217-
218-
219-
220-
221-
222-
223-
224-
225-
226-
227-
228-
229-
230-
231-
232-
233-
234-
235-
236-
237-
238-
239-
240-
241-
242-
243-
244-
245-
246-
247-
248-
249-
250-
251-
252-
253-
254-
255-
256-
257-
258-
259-
260-
261-
262-
263-
264-
265-
266-
267-
268-
269-
270-
271-
272-
273-
274-
275-
276-
277-
278-
279-
280-
281-
282-
283-
284-
285-
286-
287-
288-
289-
290-
291-
292-
293-
294-
295-
296-
297-
298-
299-
300-
301-
302-
303-
304-
305-
306-
307-
308-
309-
310-
311-
312-
313-
314-
315-
316-
317-
318-
319-
320-
321-
322-
323-
324-
325-
326-
327-
328-
329-
330-
331-
332-
333-
334-
335-
336-
337-
338-
339-
340-
341-
342-
343-
344-
345-
346-
347-
348-
349-
350-
351-
352-
353-
354-
355-
356-
357-
358-
359-
360-
361-
362-
363-
364-
365-
366-
367-
368-
369-
370-
371-
372-
373-
374-
375-
376-
377-
378-
379-
380-
381-
382-
383-
384-
385-
386-
387-
388-
389-
390-
391-
392-
393-
394-
395-
396-
397-
398-
399-
400-
401-
402-
403-
404-
405-
406-
407-
408-
409-
410-
411-
412-
413-
414-
415-
416-
417-
418-
419-
420-
421-
422-
423-
424-
425-
426-
427-
428-
429-
430-
431-
432-
433-
434-
435-
436-
437-
438-
439-
440-
441-
442-
443-
444-
445-
446-
447-
448-
449-
450-
451-
452-
453-
454-
455-
456-
457-
458-
459-
460-
461-
462-
463-
464-
465-
466-
467-
468-
469-
470-
471-
472-
473-
474-
475-
476-
477-
478-
479-
480-
481-
482-
483-
484-
485-
486-
487-
488-
489-
490-
491-
492-
493-
494-
495-
496-
497-
498-
499-
500-
501-
502-
503-
504-
505-
506-
507-
508-
509-
510-
511-
512-
513-
514-
515-
516-
517-
518-
519-
520-
521-
522-
523-
524-
525-
526-
527-
528-
529-
530-
531-
532-
533-
534-
535-
536-
537-
538-
539-
540-
541-
542-
543-
544-
545-
546-
547-
548-
549-
550-
551-
552-
553-
554-
555-
556-
557-
558-
559-
560-
561-
562-
563-
564-
565-
566-
567-
568-
569-
570-
571-
572-
573-
574-
575-
576-
577-
578-
579-
580-
581-
582-
583-
584-
585-
586-
587-
588-
589-
590-
591-
592-
593-
594-
595-
596-
597-
598-
599-
600-
601-
602-
603-
604-
605-
606-
607-
608-
609-
610-
611-
612-
613-
614-
615-
616-
617-
618-
619-
620-
621-
622-
623-
624-
625-
626-
627-
628-
629-
630-
631-
632-
633-
634-
635-
636-
637-
638-
639-
640-
641-
642-
643-
644-
645-
646-
647-
648-
649-
650-
651-
652-
653-
654-
655-
656-
657-
658-
659-
660-
661-
662-
663-
664-
665-
666-
667-
668-
669-
670-
671-
672-
673-
674-
675-
676-
677-
678-
679-
680-
681-
682-
683-
684-
685-
686-
687-
688-
689-
690-
691-
692-
693-
694-
695-
696-
697-
698-
699-
700-
701-
702-
703-
704-
705-
706-
707-
708-
709-
710-
711-
712-
713-
714-
715-
716-
717-
718-
719-
720-
721-
722-
723-
724-
725-
726-
727-
728-
729-
730-
731-
732-
733-
734-
735-
736-
737-
738-
739-
740-
741-
742-
743-
744-
745-
746-
747-
748-
749-
750-
751-
752-
753-
754-
755-
756-
757-
758-
759-
760-
761-
762-
763-
764-
765-
766-
767-
768-
769-
770-
771-
772-
773-
774-
775-
776-
777-
778-
779-
780-
781-
782-
783-
784-
785-
786-
787-
788-
789-
790-
791-
792-
793-
794-
795-
796-
797-
798-
799-
800-
801-
802-
803-
804-
805-
806-
807-
808-
809-
810-
811-
812-
813-
814-
815-
816-
817-
818-
819-
820-
821-
822-
823-
824-
825-
826-
827-
828-
829-
830-
831-
832-
833-
834-
835-
836-
837-
838-
839-
840-
841-
842-
843-
844-
845-
846-
847-
848-
849-
850-
851-
852-
853-
854-
855-
856-
857-
858-
859-
860-
861-
862-
863-
864-
865-
866-
867-
868-
869-
870-
871-
872-
873-
874-
875-
876-
877-
878-
879-
880-
881-
882-
883-
884-
885-
886-
887-
888-
889-
890-
891-
892-
893-
894-
895-
896-
897-
898-
899-
900-
901-
902-
903-
904-
905-
906-
907-
908-
909-
910-
911-
912-
913-
914-
915-
916-
917-
918-
919-
920-
921-
922-
923-
924-
925-
926-
927-
928-
929-
930-
931-
932-
933-
934-
935-
936-
937-
938-
939-
940-
941-
942-
943-
944-
945-
946-
947-
948-
949-
950-
951-
952-
953-
954-
955-
956-
957-
958-
959-
960-
961-
962-
963-
964-
965-
966-
967-
968-
969-
970-
971-
972-
973-
974-
975-
976-
977-
978-
979-
980-
981-
982-
983-
984-
985-
986-
987-
988-
989-
990-
991-
992-
993-
994-
995-
996-
997-
998-
999-
1000-

```

```

285-     if source_manifest:
286-         updated = attach_source_video_paths(specs, source_manifest)
287-         print(
288-             f"[fusion_golden] attached {updated}/{len(specs)} source video path(s) "
289-             f"from {source_manifest}",
290-             flush=True,
291-         )

```

21. user (2026-07-04T00:45:13.519Z)

```

25-     PERSON_FEATURE_NAMES,
26-     SHUTTLE_FEATURE_NAMES,
27-     SOURCE_MANIFEST_ENV,
28-     _derive_video_path,
29-     _filter_specs,
30:     attach_source_video_paths,
31-)
32-from backend.eval.golden_manifest import load_resolved_manifest
33-from backend.pipeline.audio.hit_detector import detect_hits_in_video
34-from backend.pipeline.detectors import audio_features as af
35-
36-
37-
38-
39-
40-
41-
42-
43-
44-
45-) -> List[str]:
46-     """Add the 3 audio features to each existing ``<name>_golden_features.json`` window."""
47-     specs = load_resolved_manifest(manifest_path)
48-     source_manifest = source_manifest or os.environ.get(SOURCE_MANIFEST_ENV)
49-     if source_manifest:
50-         updated = attach_source_video_paths(specs, source_manifest)
51-         print(
52-             f"[fusion_audio] attached {updated}/{len(specs)} source video path(s) "
53-             f"from {source_manifest}",
54-             flush=True,
55-         )

```

22. assistant (2026-07-04T00:45:15.599Z)

Good. Now let me check if there are any redundant calls in the main loop of `run()`:

23. user (2026-07-04T00:45:15.919Z)

```

316         for vi, spec in enumerate(specs, 1):
317             out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
318             existing = os.path.isfile(out)
319             existing_full = existing and is_full_fusion_feature_file(out)
320             # Resumable: skip a video whose full-fusion JSON already exists. A leaner
321             # windowing-only JSON is stale for A7, so re-extract it even without --fresh.
322             if missing_fusion_only and not fresh and existing_full:
323                 print(
324                     f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: full-fusion "
325                     f"features already present -> {out}",
326                     flush=True,
327                 )
328                 written.append(out)
329                 continue
330             if not fresh and existing and existing_full:
331                 print(
332                     f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already extracted "
333                     f"-> {out} (use --fresh to redo)",
334                     flush=True,
335                 )
336                 written.append(out)
337                 continue
338             if not fresh and existing and not existing_full:

```

```

339         print(
340             f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: existing JSON "
341             "is not full-fusion schema; re-extracting...",
342             flush=True,
343         )
344     print(
345         f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
346         flush=True,
347     )
348     t0 = time.monotonic()
349     if telem is not None:
350         telem.event("video_start", spec["name"])
351     try:
352         data = extract_video(
353             spec,
354             frame_skip=frame_skip,
355             iou=iou,
356             telemetry=telem,
357             use_gpu=use_gpu,
358             detector_model=detector_model,
359             max_gpu_temp_c=max_gpu_temp_c,
360         )
361     except Exception as e: # noqa: BLE001 - record machine state at death, then re-
raise
362         if telem is not None:
363             telem.event("error", spec["name"], err=str(e))
364         raise
365     n_pos = sum(c["label"] for c in data["candidates"])
366     with open(out, "w", encoding="utf-8") as f:
367         json.dump(data, f, indent=2)
368     if telem is not None:
369         telem.event(
370             "video_done",
371             spec["name"],
372             windows=len(data["candidates"]),
373             positives=n_pos,
374         )
375     print(
376         f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: DONE in
{time.monotonic() - t0:.0f}s "
377         f"| {len(data['candidates'])} windows ({n_pos} positive) |
{len(data['gts'])} golden rallies "
378         f"-> {out}",
379         flush=True,
380     )
381     written.append(out)
382     return written
383
384
385     def main() -> None:

```

24. assistant (2026-07-04T00:45:18.135Z)

Now let me check the full scope of the problem more carefully. Let me look at potential issues with `\_filter\_specs`:

25. user (2026-07-04T00:45:18.950Z)

```

120     def _filter_specs(
121         specs: Sequence[Dict[str, Any]], only: Sequence[str]
122     ) -> List[Dict[str, Any]]:
123         if not only:
124             return list(specs)
125         wanted = list(dict.fromkeys(only))
126         wanted_set = set(wanted)

```



```

127         filtered = [spec for spec in specs if str(spec.get("name")) in wanted_set]
128         found = {str(spec.get("name")) for spec in filtered}
129         missing = [name for name in wanted if name not in found]
130         if missing:
131             raise ValueError(f"--only requested unknown video(s): {' '.join(missing)}")
132         return filtered

```

26. assistant (2026-07-04T00:45:21.232Z)

Now let me verify the context around `is\_full\_fusion\_feature\_file` and check for redundant reads:

27. user (2026-07-04T00:45:21.598Z)

```

42     def is_full_fusion_feature_file(path: str) -> bool:
43         """True when an existing golden feature JSON already has shuttle+person cue dicts.
44
45         Corpus-growth runs can leave leaner windowing-only JSONs with the same filename
46         pattern. Those are useful for gts/windowing analysis, but they are not the replayable fusion
47         substrate that ``fusion_compare`` and ``ablation`` need.
48         """
49
50         try:
51             with open(path, encoding="utf-8") as f:
52                 data = json.load(f)
53         except (OSError, json.JSONDecodeError, TypeError):
54             return False
55         candidates = data.get("candidates")
56         if not isinstance(candidates, list):
57             return False
58         if not candidates:
59             return True
60         return all(
61             isinstance(candidate, dict)
62             and "shuttle" in candidate
63             and "person" in candidate
64             for candidate in candidates
65         )

```

28. assistant (2026-07-04T00:45:24.253Z)

Now let me check the audio file for similar patterns:

29. user (2026-07-04T00:45:24.602Z)

```

39     def augment(
40         manifest_path: str,
41         features_dir: str,
42         k: float = 2.5,
43         source_manifest: Optional[str] = None,
44         only: Sequence[str] = (),
45     ) -> List[str]:
46         """Add the 3 audio features to each existing ``<name>_golden_features.json``
47         window."""
48         specs = load_resolved_manifest(manifest_path)
49         source_manifest = source_manifest or os.environ.get(SOURCE_MANIFEST_ENV)
50         if source_manifest:
51             updated = attach_source_video_paths(specs, source_manifest)
52             print(
53                 f"[fusion_audio] attached {updated}/{len(specs)} source video path(s) "
54                 f"from {source_manifest}",
55                 flush=True,
56             )

```

```

56         specs = _filter_specs(specs, only)
57         written: List[str] = []
58         for spec in specs:
59             jp = os.path.join(features_dir, f"{spec['name']}_golden_features.json")
60             if not os.path.isfile(jp):
61                 print(
62                     f"[fusion_audio] {spec['name']}: no feature JSON ? run fusion_golden
first; skipping"
63                 )
64                 continue
65             video = _derive_video_path(spec)
66             if not os.path.isfile(video):
67                 raise FileNotFoundError(
68                     f"{spec.get('name')}: source video not found for audio augmentation:
{video}"
69                 )
70             print(
71                 f"[fusion_audio] {spec['name']}: detecting audio hits (k={k})...",
72                 flush=True,
73             )
74             hits = detect_hits_in_video(video, k=k)
75             with open(jp, encoding="utf-8") as f:
76                 data = json.load(f)
77             for c in data["candidates"]:
78                 c["audio"] = af.compute_audio_features(hits, c["start"], c["end"])
79             with open(jp, "w", encoding="utf-8") as f:
80                 json.dump(data, f, indent=2)
81             print(
82                 f"[fusion_audio] {spec['name']}: {len(hits)} hits over
{len(data['candidates'])} windows",
83                 flush=True,
84             )
85             written.append(jp)
86         return written
87
88

```

30. assistant (2026-07-04T00:45:42.218Z)

Now I have a complete picture. Let me analyze the key efficiency issues:

Analysis:

1. ****Line 319 in fusion_golden.py - ``is_full_fusion_feature_file()`` double-read****: The function reads+parses the entire feature JSON (line 51-52) just to check if it has "shuttle" and "person" keys in candidates. This is called once per video in the loop (line 319), and the feature JSON can be large for a full match. However, looking at the actual flow (lines 316-337), the file is ONLY read once for schema validation? it's not re-read elsewhere for actual feature extraction. The ``extract_video()`` function creates a new JSON from scratch. So while expensive for large JSONs, it's a one-read-per-video operation.

2. ****Line 299-300 in fusion_golden.py - Double stat call in lambda****: When ``smallest_first=True``, the code calls both ``os.path.isfile()`` AND ``os.path.getsize()`` on the trajectory file. This is O(n) where n is the number of specs, but it's in a sort lambda (happens once, not per-iteration). Each file gets two stat calls instead of one (isfile check + size check could be combined). This is measurable redundancy.

3. ****Line 106 in fusion_golden.py - ``load_source_video_paths()`` in ``attach_source_video_paths()``****: This reads and parses the entire source manifest JSON, then iterates once through specs. Called once per run (line 286 in fusion_golden.py, line 50 in fusion_audio.py). No loop overhead? acceptable pattern.

4. ****Lines 128-129 in fusion_golden.py - ``_filter_specs()`` redundancy****: After filtering, it rebuilds ``found`` set (line 128) from ``filtered`` list by re-extracting names via

`str(spec.get("name"))` again. Already computed these names in line 127's list comprehension. Minor but unnecessary recomputation.

5. ****Lines 60, 66 in fusion_audio.py - Double isfile checks per spec****: In the `augment()` loop, line 60 checks if the feature JSON exists, and line 66 checks if the video exists. Both files will be opened/read immediately after (lines 75-76 for JSON), so the isfile checks are reasonable guards, not redundant.

6. ****Line 365 in fusion_golden.py - Single pass through candidates****: Computing `n\_pos = sum(c["label"] for c in data["candidates"])` is a single iteration?not redundant since candidates haven't been iterated before.

Measurable waste candidates:

```
```json
[
 {
 "file": "backend/eval/fusion_golden.py",
 "line": 299,
 "summary": "os.path.isfile() + os.path.getsize() double-stat the trajectory file in sort
lambda when smallest_first=True; could combine into one operation or cache results",
 "failure_scenario": "With hundreds of videos and smallest_first=True, this causes 2x file
stat syscalls vs 1x on line 300 alone"
 },
 {
 "file": "backend/eval/fusion_golden.py",
 "line": 128,
 "summary": "_filter_specs() recomputes spec names via str(spec.get('name')) in set
comprehension (line 128) after already computing them in list comprehension (line 127); names
extracted twice",
 "failure_scenario": "With thousands of specs, this is O(n) redundant string conversions;
small but measurable overhead"
 },
 {
 "file": "backend/eval/fusion_golden.py",
 "line": 42,
 "summary": "is_full_fusion_feature_file() reads entire feature JSON into memory just to
check schema keys; with ~500+ candidate windows per video, JSON can be multi-MB for a full
match",
 "failure_scenario": "Called once per video in run() loop (line 319). If corpus has 100
videos x 2-5MB JSONs = 200-500MB unnecessary reads to validate schema, but only during skip-
check, not extraction phase"
 }
]
```
```

```
```json
[
 {
 "file": "backend/eval/fusion_golden.py",
 "line": 299,
 "summary": "os.path.isfile() + os.path.getsize() double-stat the trajectory file in sort
lambda; both call filesystem with smallest_first=True",
 "failure_scenario": "With hundreds of videos, causes 2x syscalls per video instead of 1x;
getsize() internally checks if file exists, making the explicit isfile() call redundant"
 },
 {
 "file": "backend/eval/fusion_golden.py",
 "line": 128,
 "summary": "_filter_specs() line 128 rebuilds set of names via str(spec.get('name')) after
line 127 already extracted them in list comp; recomputes strings for every spec",
 "failure_scenario": "With large spec lists, O(n) redundant string conversions; could reuse
found set from filtered list comprehension"
 },
 {

```

```

 "file": "backend/eval/fusion_golden.py",
 "line": 42,
 "summary": "is_full_fusion_feature_file() parses entire feature JSON to validate schema
(shuttle+person keys present); large JSONs (multi-MB for full-match windows) loaded just for
boolean check",
 "failure_scenario": "Line 319 calls this once per video during run(); if skipped, JSON isn't
re-read, but validation reads large file unnecessarily (could peek at first candidate only or
use lazy check)"
 }
]
...

```

<json>

```

[
 {
 "file": "backend/eval/fusion_golden.py",
 "line": 299,
 "summary": "os.path.isfile() + os.path.getsize() double-stat trajectory file in sort lambda
when smallest_first=True; getsize() internally checks file existence, making explicit isfile()
redundant",
 "failure_scenario": "With hundreds of videos, causes 2x syscalls per video; could call
getsize() once and catch exception if file missing"
 },
 {
 "file": "backend/eval/fusion_golden.py",
 "line": 128,
 "summary": "_filter_specs() recomputes spec names via str(spec.get('name')) in set
comprehension (line 128) after already extracting them in list comprehension (line 127)",
 "failure_scenario": "With large spec lists, O(n) redundant string conversions on every
filtered spec; could reuse names from filtered list"
 },
 {
 "file": "backend/eval/fusion_golden.py",
 "line": 42,
 "summary": "is_full_fusion_feature_file() parses entire feature JSON (potentially multi-MB
with 500+ windows) just to check presence of shuttle+person keys; full file loaded into memory
for boolean check",
 "failure_scenario": "Line 319 calls this once per video; if file is large, unnecessary
memory pressure; could check only first candidate or defer deep validation"
 }
]
</json>

```